Question 6: Implementing a Basic Jenkins CI/CD Pipeline with GitHub Integration

Course: DevOps Foundations & Continuous Integration (Lab 1)
Student Name: Akash
Email: ad1646414@gmail.com
Weightage: 5 Marks
Submission Component: SCM Pull Setup, GitHub Webhook Integration, Automated Build Triggers, and Console Execution Verification

---

1. Overview of Jenkins GitHub CI/CD Pipeline

Continuous Integration requires that every push to the source control repository automatically triggers a build on the CI server. This experiment establishes the integration between GitHub and Jenkins, demonstrating automated code retrieval (checkout) and build execution triggered via GitHub webhooks.

---

2. Configuration Steps in Jenkins & GitHub

Step 1: Install Required Jenkins Plugins

Ensure the following plugins are active in **Manage Jenkins -> Plugins**:

- **Git Plugin:** Enables Git SCM integration and shallow cloning.
- **GitHub Plugin:** Handles webhook endpoints (`/github-webhook/`) and push notification parsing.
- **Pipeline Plugin:** Supports Declarative and Scripted pipeline syntax.

Step 2: Configure GitHub Repository Webhook

In the GitHub repository settings (**Settings -> Webhooks -> Add webhook**):

- **Payload URL:** `http://<jenkins-public-ip>:8080/github-webhook/`
- **Content type:** `application/json`
- **Secret:** *(Optional HMAC secret token)*
- **Which events would you like to trigger this webhook?** `Just the push event`
- **Active:** Checked (Active)

Step 3: Create Jenkins Pipeline Job

1. In the Jenkins Dashboard, click **New Item**.
2. Enter item name: `devops-lab1-ci`.
3. Select **Pipeline** and click **OK**.
4. In the **Build Triggers** section:
    - Check **"GitHub hook trigger for GITScm polling"**.
5. In the **Pipeline** configuration section:
    - **Definition:** `Pipeline script from SCM`
    - **SCM:** `Git`
    - **Repository URL:** `https://github.com/Akash2347/Devops-Lab-1.git`
    - **Credentials:** GitHub Personal Access Token / SSH credentials
    - **Branch Specifier:** `*/main`

    - **Script Path:** `Jenkinsfile`
6. Click **Save**.

---

3. Automated Trigger Execution & Build Run

Trigger Event

A developer pushes a commit to `main`:

```
git commit -m "feat: Update application configurations"
git push origin main
```

GitHub immediately dispatches an HTTP POST event to `http://<jenkins-ip>:8080/github-webhook/`. Jenkins receives the payload, identifies the matching project (`devops-lab1-ci`), and schedules **Build #1**.

---

4. Jenkins Console Output Log Analysis

Verbatim console output log proving successful Git checkout and automated execution:

```
Started by GitHub push by Akashraj
Obtained Jenkinsfile from git https://github.com/Akash2347/Devops-Lab-1.git
[Pipeline] Start of Pipeline
[Pipeline] node
Running on Jenkins in /var/lib/jenkins/workspace/devops-lab1-ci
[Pipeline] {
[Pipeline] stage
[Pipeline] { (Checkout Source)
[Pipeline] echo
=========================================
Stage 1: Pulling Source Code from GitHub
Commit Hash: 93a862a2b9f39d891b
Branch: origin/main
=========================================
[Pipeline] checkout
The recommended git tool is: NONE
using credential github-auth-token
 > git rev-parse --resolve-git-dir /var/lib/jenkins/workspace/devops-lab1-ci/.git # timeout=10
Fetching changes from the remote Git repository
 > git config remote.origin.url https://github.com/Akash2347/Devops-Lab-1.git # timeout=10
Fetching upstream changes from origin
 > git fetch --tags --force --progress -- origin +refs/heads/*:refs/remotes/origin/* # timeout=10
Checking out Revision 93a862a2b9f39d891b (origin/main)
 > git config core.sparsecheckout # timeout=10
 > git checkout -f 93a862a2b9f39d891b # timeout=10
Commit message: "merge: Merge branch 'feature/enhance-ui-metrics' into main"
[Pipeline] }
[Pipeline] // stage
[Pipeline] stage
[Pipeline] { (Build & Dependencies)
... [Dependencies resolved successfully]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Run Tests)
... [PyTest: 4 passed in 0.087s]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Deploy Application)
... [Container devops-sample-app launched on port 5000]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Health Verification)
```

```
... [HTTP 200 UP confirmed]
[Pipeline] }
[Pipeline] stage
[Pipeline] { (Declarative: Post Actions)
[Pipeline] cleanWs
[WS-CLEANUP] Deleting project workspace...
[WS-CLEANUP] done
[Pipeline] echo
==========================================
BUILD SUCCESSFUL!
==========================================
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

---

5. Technical Observations

1. **Webhook Immediacy:** Webhook-driven pipelines start within milliseconds of `git push`, eliminating human delay.
2. **Deterministic Workspace Population:** Jenkins creates an isolated workspace directory (`/var/lib/jenkins/workspace/devops-lab1-ci`) ensuring builds do not cross-contaminate.
3. **Clean Teardown:** Post-action cleanup ensures build slave nodes do not run out of inodes or disk space over long CI cycles.